Actividad: Programación de Solución Robótica

Grupo 3: Mateo Flores | Federico Soler

En algunos de los lenguajes disponibles en las plataformas de programación robótica libres y abiertas o con licencia de estudiante (CoppeliaSim, Webots, Arduino, etc.) realizar un programa en los lenguajes permitidos (Phyton, C, C++, Java, lenguaje privativo de la marca del robot, etc.) que realice el movimiento de un robot. Explicando a qué parte del robot y de qué manera se producirá ese movimiento. Contemplar la posibilidad de verificación y simulación del movimiento del robot con el software utilizado.

Simulación: https://wokwi.com/projects/478894577015484417
Github: https://github.com/federicosolerup/brazo-robotico-arduino-wokwi

---

Wokwi: entorno web que permite simular una placa Arduino UNO que ejecuta instrucciones escritas en C++.

Arduino: es una placa electrónica (hardware) que permite controlar dispositivos físicos, como motores, sensores, luces y robots desde un entorno de programación (software).

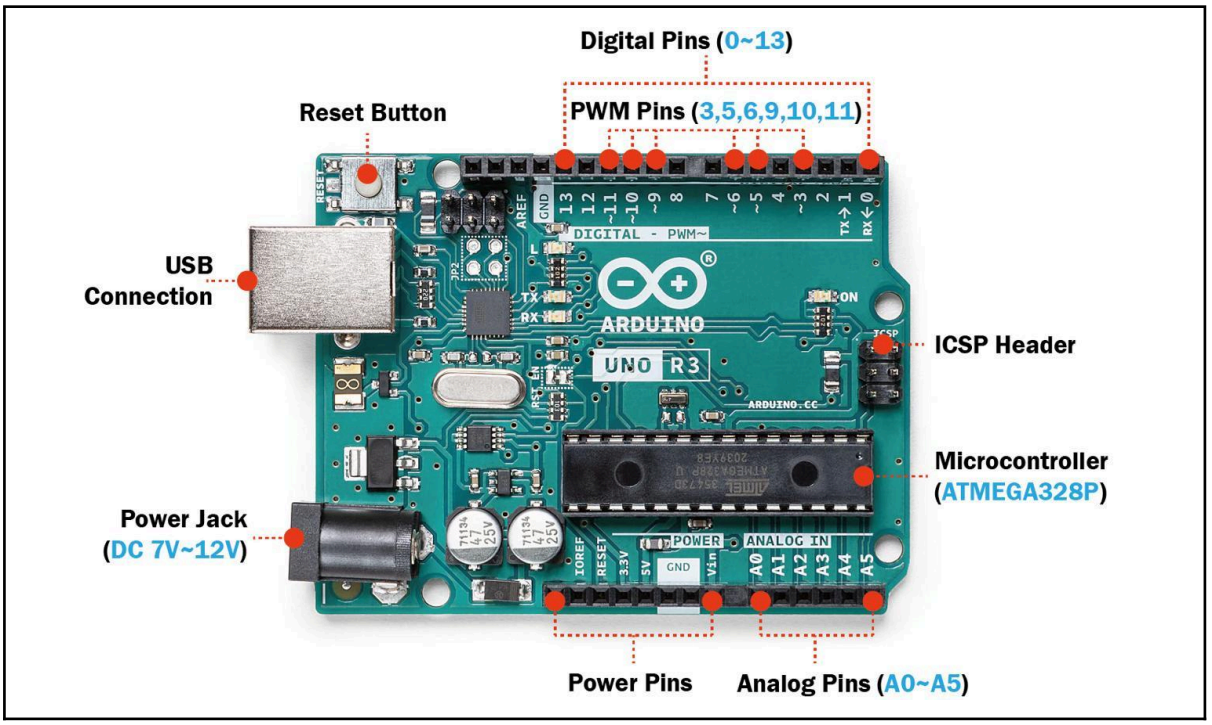


Figura 1 - Arduino UNO, la placa que usamos en nuestro proyecto.

Para este proyecto programamos en C++ un brazo robótico de dos grados de libertad, compuesto por hombro, húmero, codo y antebrazo. La muñeca, sería el efector final y las articulaciones del hombro y codo se movilizan mediante dos servomotores.

Secuencia: En cada tramo se modifica únicamente el ángulo de una articulación, mientras la otra mantiene su consigna constante.

Configuración	Hombro	Codo	Movimiento para llegar
P0 · Reposo inicial	30°	30°	Ninguno
P1	75°	30°	Hombro: 30° → 75°
P2	75°	90°	Codo: 30° → 90°
P3	30°	90°	Hombro: 75° → 30°
P4 · Regreso al reposo	30°	30°	Codo: 90° → 30°

Trayectoria: para obtener un movimiento suave, se aplica por separado a cada articulación un polinomio de tercer grado. Usamos la misma fórmula cuatro veces, pero cambiamos los ángulos inicial y final y el motor que recibe las órdenes. La fórmula simplemente calcula qué ángulo le corresponde al servomotor en cada instante.

$$q(t) = q_0 + 3(q_f - q_0)(t/T)^2 - 2(q_f - q_0)(t/T)^3$$

donde $q(t)$ es el ángulo de la articulación activa en t, q_0 es el ángulo inicial, q_f el ángulo final, t el tiempo transcurrido, T la duración total del movimiento, y $q_f - q_0$ cuánto girar.

Aplicando estas condiciones a $q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$ se obtienen los siguientes coeficientes que determinan la forma de la curva.

$$a_0 = q_0$$
$$a_1 = 0$$
$$a_2 = 3(qf - q_0)/T^2$$
$$a_3 = -2(qf - q_0)/T^3$$

Esos valores concretos permiten que la curva cumpla cuatro condiciones de borde. Una condición de borde es algo que exigimos que ocurra al comienzo o al final del movimiento.

Condición	Qué significa
$q(0) = q_0$	El hombro empieza en 30°
$q(T) = q_f$	El hombro termina en 75°
$\dot{q}(0) = 0$	Empieza con velocidad angular cero
$\dot{q}(T) = 0$	Termina con velocidad angular cero

Código (C++): se utiliza la biblioteca Servo para controlar el hombro mediante D9 y el codo mediante D10. Durante la inicialización, posiciona primero el hombro en 30°, espera 1,5 segundos y luego posiciona el codo en 30°, con otra espera de 1,5 segundos. Al presionar el pulsador conectado a D2, se ejecutan cuatro movimientos: elevar el hombro, flexionar el codo, devolver el hombro a su posición inicial y extender nuevamente el codo.

```
#include <Servo.h>

Servo servoHombro;
Servo servoCodo;
const int PIN_BOTON = 2;
const unsigned long T = 1500;
const unsigned long PASO_MS = 20;
const int PUNTOS[5][2] = {{30, 30}, {75, 30},
{75, 90}, {30, 90}, {30, 30}};
int anguloHombro = 30;
int anguloCodo = 30;
unsigned int ciclo = 0;
unsigned long inicioCiclo;

void registrar(const char *fase) {
  Serial.print(ciclo);
  Serial.print(',');
  Serial.print(millis() - inicioCiclo);
  Serial.print(',');
  Serial.print(anguloHombro);
  Serial.print(',');
  Serial.print(anguloCodo);
  Serial.print(',');
  Serial.println(fase);
}

void setup() {
  pinMode(PIN_BOTON, INPUT_PULLUP);
  servoHombro.attach(9);
  servoHombro.write(anguloHombro);
  delay(T);
  servoCodo.attach(10);
  servoCodo.write(anguloCodo);
  delay(T);
  Serial.begin(115200);

Serial.println("ciclo,tiempo_ms,hombro_deg,co
do_deg,fase");
  Serial.println("# LISTO: presionar el
pulsador");
}
```

```
void moverUnaArticulacion(Servo &motor, int
&anguloActual, int destino,
                          const char *fase) {
  const int origen = anguloActual;
  unsigned long inicio = millis();
  float s = 0;
  while (s < 1.0) {
    s = (millis() - inicio) / (float)T;
    if (s > 1.0) s = 1.0;
    float k = 3*s*s - 2*s*s*s;
    anguloActual = origen + (destino - origen) * k;
    motor.write(anguloActual);
    registrar(fase);
    delay(PASO_MS);
  }
}

void pausaConRegistro(const char *fase) {
  const unsigned long duracion = 300;
  unsigned long inicio = millis();
  while (millis() - inicio < duracion) {
    registrar(fase);
    unsigned long transcurrido = millis() - inicio;
    if (transcurrido < duracion) {
      unsigned long restante = duracion -
transcurrido;
      delay(restante < PASO_MS ? restante :
PASO_MS);
    }
  }
  registrar(fase);
}

void loop() {
  if (digitalRead(PIN_BOTON) == LOW) {
    ciclo++;
    inicioCiclo = millis();
    registrar("inicio");
    moverUnaArticulacion(servoHombro, anguloHombro,
PUNTOS[1][0], "hombro_1");
    pausaConRegistro("pausa_1");
    moverUnaArticulacion(servoCodo, anguloCodo,
PUNTOS[2][1], "codo_2");
    pausaConRegistro("pausa_2");
    moverUnaArticulacion(servoHombro, anguloHombro,
PUNTOS[3][0], "hombro_3");
    pausaConRegistro("pausa_3");
    moverUnaArticulacion(servoCodo, anguloCodo,
PUNTOS[4][1], "codo_4");
    pausaConRegistro("pausa_4");
    registrar("fin");
    Serial.println("# FIN_CICLO");
    while (digitalRead(PIN_BOTON) == LOW) delay(10);
  }
}
```

Conexionado: El Arduino UNO tiene una serie de pines para comunicarse con componentes electrónicos. En nuestro proyecto, la placa recibe la señal del pulsador y envía órdenes a los dos servomotores que controlan el hombro y el codo. Además de las señales, ambos servos reciben 5V y GND.

5V es una conexión que proporciona una tensión de 5 voltios respecto de GND. En nuestro proyecto de Wokwi, alimenta los servomotores. GND significa Ground (tierra o masa). Es el punto de referencia de 0 voltios del circuito y permite establecer un camino de retorno para la corriente.

En Wokwi podemos alimentar los dos servos desde el Arduino. En un montaje físico, sin embargo, suele convenir una fuente de alimentación externa adecuada para los motores, compartiendo GND con Arduino, porque los servos pueden consumir más corriente de la que la placa puede suministrar con seguridad.

D9 y D10 son independientes, podemos ordenar que uno cambie de posición mientras el otro conserva su ángulo. D2 es una entrada digital para detectar cuándo presionamos el botón para iniciar el movimiento. Al presionar el pulsador, conectamos D2 con GND y Arduino detecta el cambio.

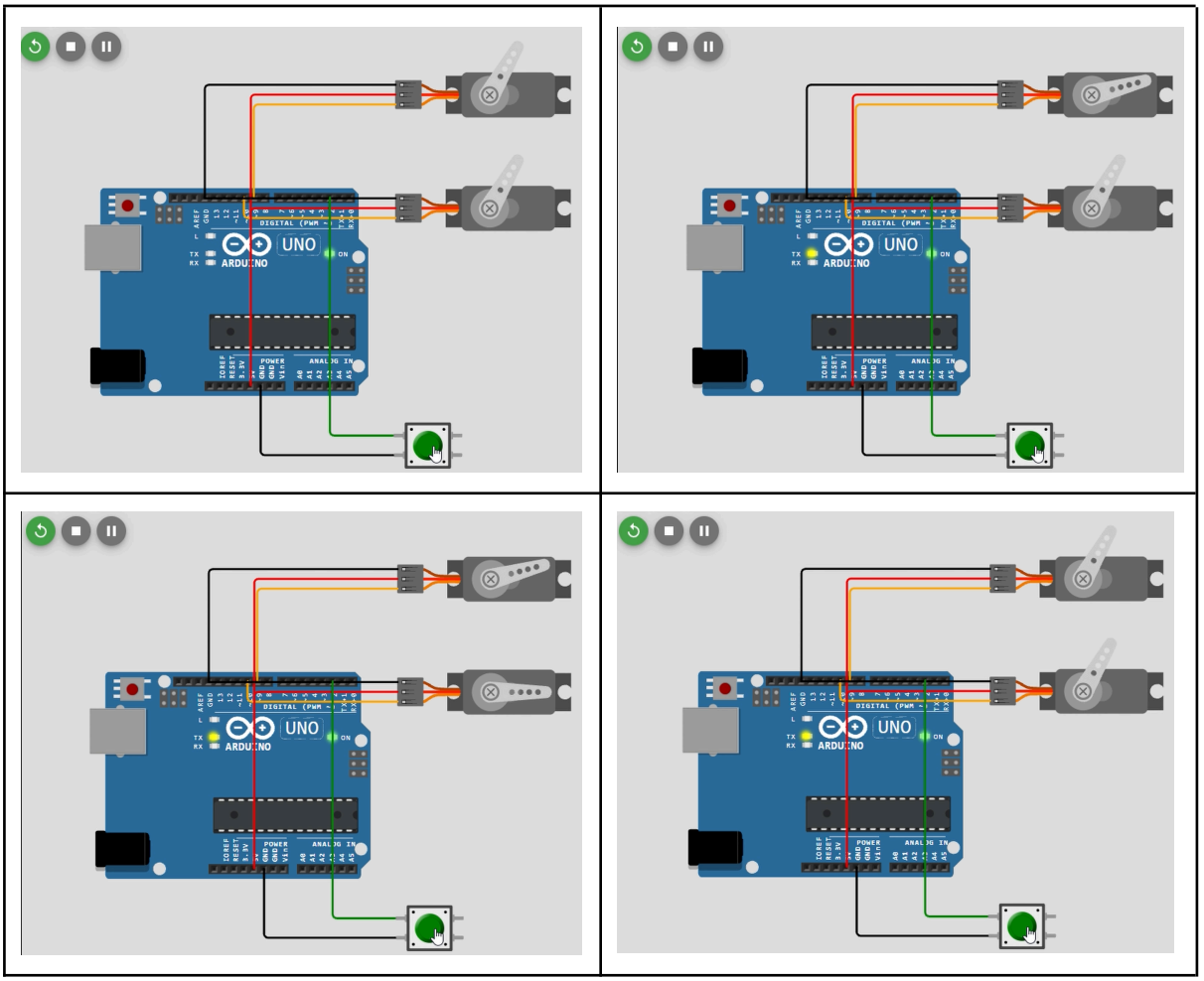


Figura 2. Simulación en Wokwi de las cuatro configuraciones del brazo robótico, mediante el control independiente de los servomotores del hombro y del codo.

Visualización: Dado que Wokwi no provee brazos robóticos como componentes, se desarrolló un notebook en Python que se ejecuta en Google Colab para visualizar el movimiento del brazo. El programa utiliza el registro CSV generado en Wokwi, con los tiempos y las consignas angulares enviadas a los servomotores, y calcula mediante cinemática directa las posiciones del codo y de la muñeca.

Para la representación gráfica se utilizó Matplotlib, junto con su función FuncAnimation, que permite actualizar las posiciones del húmero, el antebrazo y la muñeca en cada cuadro. También se muestran los ángulos articulares en gráficos sincronizados con la animación. Se eligió esta librería porque permite combinar el dibujo del brazo y los gráficos en una misma figura, funciona en Google Colab y facilita exportar el resultado como video, GIF o reproductor HTML lo cuál nos va a asistir en nuestras explicaciones el día de la presentación.

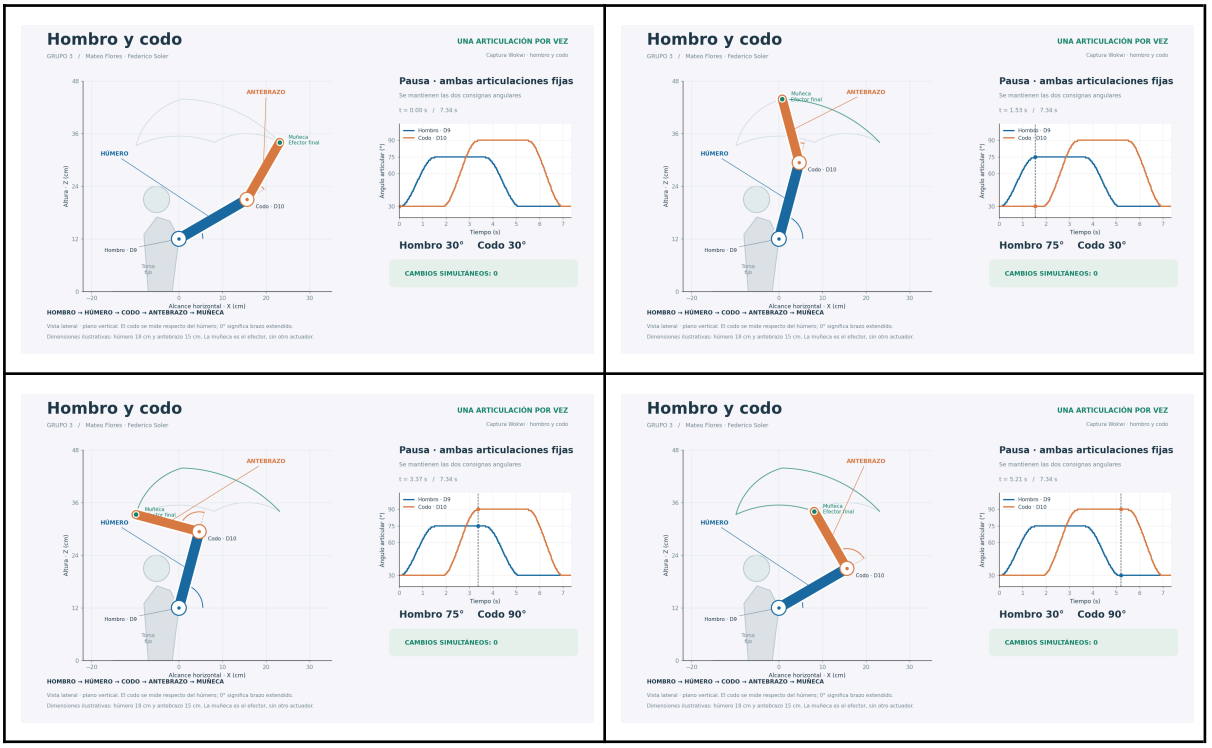


Figura 3. Visualización cinemática del brazo en Python, con representación de las articulaciones y gráficos de los ángulos durante la secuencia.